

Decision Control Structures in C++ (if-else, switch, ternary)

📌 Definition to Remember

Decision control structures in C++ enable conditional branching using if-else ladders, switch statements with jump tables, and ternary conditional operators.

1. Concept Overview & Fundamentals 🧠

Decision Control Structures in C++ (if-else, switch, ternary) provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

- **Zero-Overhead Principle:** What you don't use, you don't pay for; what you do use, you couldn't hand code any better.
- **Type Safety & Streams:** Type-safe stream I/O (`std::cin` , `std::cout`) prevents runtime format mismatch vulnerabilities.
- **Resource Acquisition Is Initialization (RAII):** Automatic resource management via constructors, destructors, and smart pointers.

2. Technical Architecture & Memory Mechanics ⚙️

C++ Runtime Execution Architecture

```
+-----+
|                STACK SEGMENT                |
| - Local scope objects, references, RAII stack frames |
| - Deterministic automatic destruction upon scope exit |
+-----+
|
| v
| ^
|
+-----+
|                HEAP / FREE STORE                |
| - Dynamic objects allocated via new, std::vector, etc. |
| - Managed via delete / smart pointers |
+-----+
|                STATIC & GLOBAL MEMORY                |
| - Global objects, static class variables, string literals |
+-----+
|                CODE / TEXT SEGMENT                |
| - Executable machine code, inlined functions, templates |
+-----+
```

Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>std::cin</code> / <code>std::cout</code>	<code>scanf</code> / <code>printf</code>	Type-safe, extensible for custom classes	Buffered I/O (optimizable via <code>ios::sync_with_stdio(0)</code>)
References (<code>Type&</code>)	Pointers (<code>Type*</code>)	Non-nullable, clean syntax without dereferencing	Direct alias; zero runtime overhead
<code>std::vector<T></code>	Raw Dynamic Array (<code>malloc</code>)	Automatic resizing, bounds safety, memory management	Amortized O(1) push_back, contiguous cache
<code>nullptr</code>	<code>NULL</code> / <code>0</code>	Strongly typed pointer literal preventing integer overload bugs	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Decision Control Structure (if-else, switch, goto, ternary)" <<
    "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- **Quantitative Finance & HFT:** High-frequency trading platforms use modern C++ for microsecond-level execution.
- **Game Engine Architecture:** Unreal Engine, Unity Core, and AAA game engines utilize C++ for real-time 3D rendering.
- **Deep Learning Frameworks:** Core tensor compute backends (PyTorch, TensorFlow, ONNX Runtime) are written in C++ and CUDA.
- **Web Browsers & Compilers:** Google Chrome (V8 engine), LLVM/Clang, and GCC are implemented in C++.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: AbsoluteTO

Problem Description & Objective:

Implementation of AbsoluteTO

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to calculate the absolute value of a given integer using ternary operator.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Ternary Operator (`condition ? val1 : val2`).
 *
 * 2. WHY IT IS USED?
 *    - Provides a concise, single-line inline conditional expression instead of `if-else`.
 *
 * 3. HOW IT IS USED?
 *    - `int absValue = (number < 0) ? -number : number;`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Read `number`.
 *    * Step 2: Check if `number < 0`. If true, return `-number`; else return `number`.
 *    * Step 3: Output `absValue`.
 *    =====
 */

#include <iostream> // Header for console stream I/O

using namespace std; // Standard namespace

// --- Main Execution Entry Point ---
int main() { // Program start
    // Declare integer variable
    int number; // Stores input integer

    // Prompt user for input integer
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> number;

    // Evaluate absolute value using ternary conditional operator
    // Condition: (number < 0) -> If true return -number, else return number
    int absValue = (number < 0) ? -number : number;

    // Output calculated absolute value
    cout << "Absolute value of " << number << " is: " << absValue << endl;

    // Return zero
    return 0;
}

```

Problem 2: CategorizeAgeGroup

Problem Description & Objective:

Implementation of CategorizeAgeGroup

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that categorize a person into different age groups.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Multi-branch Conditional Ladder (`if ... else if ... else`).
 *
 * 2. WHY IT IS USED?
 *    - Evaluates mutually exclusive numerical ranges sequentially.
 *
 * 3. HOW IT IS USED?
 *    - Tests conditions `age < 0`, `age < 13`, `age < 20`, `age < 60`, `else`.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input `age`.
 *   Step 2: Evaluate range conditions top-down.
 *   Step 3: Output appropriate category label.
 * =====
 */

#include <iostream> // Console I/O stream header

using namespace std; // Standard namespace

// --- Main Execution Entry Point ---
int main() { // Main entry point
    // Declare integer variable for age
    int age;

    // Prompt for user age
    cout << "Enter age: ";
    // Read user input from standard input stream
    cin >> age;

    // Branch 1: Invalid negative age check
    if (age < 0) {
        // Output formatted data to console stream
        cout << "Invalid age entered!" << endl;
    }
    // Branch 2: Child category (< 13 years)
    else if (age < 13) {
        // Output formatted data to console stream
        cout << "Category: Child" << endl;
    }
    // Branch 3: Teenager category (< 20 years)
    else if (age < 20) {
        // Output formatted data to console stream
        cout << "Category: Teen" << endl;
    }
    // Branch 4: Adult category (< 60 years)
    else if (age < 60) {
        // Output formatted data to console stream

```

```
        cout << "Category: Adult" << endl;
    }
    // Branch 5: Senior citizen category (>= 60 years)
    else {
        // Output formatted data to console stream
        cout << "Category: Senior" << endl;
    }

    // Return 0 indicating clean finish
    return 0;
}
```

Problem 3: GradeCalculator

Problem Description & Objective:

Implementation of GradeCalculator

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that calculates grades based on marks.
 * A -> above 90%
 * B -> above 75%
 * C -> above 60%
 * D -> above 30%
 * F -> below 30%
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Cascading Relational Range Comparisons (`if-else if`).
 *
 * 2. WHY IT IS USED?
 *    - Maps numerical percentage scores to letter grades.
 *
 * 3. HOW IT IS USED?
 *    - Compares `percentage` against thresholds 90, 75, 60, 30.
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Read `percentage`.
 *    Step 2: Test score against grade cutoffs from highest to lowest.
 *    Step 3: Display calculated Grade letter.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    double percentage;

    // Output formatted data to console stream
    cout << "Enter student percentage (0-100): ";
    // Read user input from standard input stream
    cin >> percentage;

    // Evaluate conditional decision logic
    if (percentage > 90) {
        // Output formatted data to console stream
        cout << "Grade: A" << endl;
    } else if (percentage > 75) {
        // Output formatted data to console stream
        cout << "Grade: B" << endl;
    } else if (percentage > 60) {
        // Output formatted data to console stream
        cout << "Grade: C" << endl;
    } // Evaluate conditional decision logic
}

```

```
} else if (percentage >= 30) {  
    // Output formatted data to console stream  
    cout << "Grade: D" << endl;  
    // Alternative conditional branch  
} else {  
    // Output formatted data to console stream  
    cout << "Grade: F" << endl;  
}  
  
// Return zero code indicating successful program finish  
return 0;  
}
```

Problem 4: GreatestNumber

Problem Description & Objective:

Implementation of GreatestNumber

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that determines the greatest of the three numbers.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Logical AND Operator (`&&`) in Decision Control.
 *
 * 2. WHY IT IS USED?
 *    - Compound condition `n1 >= n2 && n1 >= n3` checks if a number is greater than both peers
simultaneously.
 *
 * 3. HOW IT IS USED?
 *    - Evaluates 3 numbers with `&&` comparison logic.
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Input three numbers `n1, n2, n3`.
 *    Step 2: If `n1 >= n2` and `n1 >= n3`, `n1` is greatest.
 *    Step 3: Else if `n2 >= n1` and `n2 >= n3`, `n2` is greatest.
 *    Step 4: Else `n3` is greatest.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n1, n2, n3;

    // Output formatted data to console stream
    cout << "Enter three numbers: ";
    // Read user input from standard input stream
    cin >> n1 >> n2 >> n3;

    // Evaluate conditional decision logic
    if (n1 >= n2 && n1 >= n3) {
        // Output formatted data to console stream
        cout << "Greatest number is: " << n1 << endl;
    } else if (n2 >= n1 && n2 >= n3) {
        // Output formatted data to console stream
        cout << "Greatest number is: " << n2 << endl;
    } else {
        // Alternative conditional branch
        // Output formatted data to console stream
        cout << "Greatest number is: " << n3 << endl;
    }

    // Return zero code indicating successful program finish

```

```
    return 0;  
}
```

Problem 5: LeapYear

Problem Description & Objective:

Implementation of LeapYear

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that determines if a given year is a leap year
 * (considering conditions like divisible by 4 but not 100, unless also divisible by 400).
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Compound Logical Expressions (`||`, `&&`, `%`).
 *
 * 2. WHY IT IS USED?
 *    - Gregorian calendar leap year rule requires modulo 400 OR (modulo 4 AND NOT modulo 100).
 *
 * 3. HOW IT IS USED?
 *    - `(year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Input `year`.
 *    Step 2: Evaluate compound leap year condition.
 *    Step 3: Print result message.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int year;

    // Output formatted data to console stream
    cout << "Enter a year: ";
    // Read user input from standard input stream
    cin >> year;

    // Evaluate conditional decision logic
    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
        // Output formatted data to console stream
        cout << year << " is a Leap Year." << endl;
    } else {
        // Alternative conditional branch
        // Output formatted data to console stream
        cout << year << " is NOT a Leap Year." << endl;
    }

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 6: MinimumNumberTO

Problem Description & Objective:

Implementation of MinimumNumberTO

C++ Code Implementation:

```
/*
 * Question:
 * Create a program to find the minimum of two numbers using ternary operator.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Ternary Minimum Expression.
 *
 * 2. WHY IT IS USED?
 *    - Simplifies minimum value selection.
 *
 * 3. HOW IT IS USED?
 *    - `int minVal = (a < b) ? a : b;`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Read two integers `a` and `b`.
 *    Step 2: Assign `a` to `minVal` if `a < b`, else assign `b`.
 *    Step 3: Output `minVal`.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int a, b;

    // Output formatted data to console stream
    cout << "Enter two integers: ";
    // Read user input from standard input stream
    cin >> a >> b;

    int minVal = (a < b) ? a : b;

    // Output formatted data to console stream
    cout << "Minimum number is: " << minVal << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 7: NumberChecker

Problem Description & Objective:

Implementation of NumberChecker

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that determines if a number is positive, negative, or zero.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Three-Way Branching (`if`, `else if`, `else`).
 *
 * 2. WHY IT IS USED?
 *    - Categorizes numbers relative to 0.
 *
 * 3. HOW IT IS USED?
 *    - `num > 0` (Positive), `num < 0` (Negative), `else` (Zero).
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Read `num`.
 *    Step 2: Check sign.
 *    Step 3: Print result.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int num;

    // Output formatted data to console stream
    cout << "Enter a number: ";
    // Read user input from standard input stream
    cin >> num;

    // Evaluate conditional decision logic
    if (num > 0) {
        // Output formatted data to console stream
        cout << num << " is Positive." << endl;
    } else if (num < 0) {
        // Output formatted data to console stream
        cout << num << " is Negative." << endl;
    } else {
        // Alternative conditional branch
        // Output formatted data to console stream
        cout << "The number is Zero." << endl;
    }

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 8: OddEven

Problem Description & Objective:

Implementation of OddEven

C++ Code Implementation:

```

/*
 * Question:
 * Create a program that determines if a number is odd or even.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Modulo Operator (`% 2`).
 *
 * 2. WHY IT IS USED?
 *    - Remainder of division by 2 determines parity: 0 -> Even, 1 -> Odd.
 *
 * 3. HOW IT IS USED?
 *    - `if (number % 2 == 0)`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Read integer `number`.
 *    Step 2: Check `number % 2`.
 *    Step 3: Print Even or Odd.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int number;

    // Output formatted data to console stream
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> number;

    // Evaluate conditional decision logic
    if (number % 2 == 0) {
        // Output formatted data to console stream
        cout << number << " is Even." << endl;
    } else {
        // Output formatted data to console stream
        cout << number << " is Odd." << endl;
    }

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 9: OddEvenTO

Problem Description & Objective:

Implementation of OddEvenTO

C++ Code Implementation:

```
/*
 * Question:
 * Create a program to find if the given number is even or odd using ternary operator.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Ternary Parity Check returning a `std::string`.
 *
 * 2. WHY IT IS USED?
 *    - Demonstrates returning string literals directly from a ternary expression.
 *
 * 3. HOW IT IS USED?
 *    - `string result = (number % 2 == 0) ? "Even" : "Odd";`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Input `number`.
 *    Step 2: Evaluate ternary expression.
 *    Step 3: Print string result.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int number;

    // Output formatted data to console stream
    cout << "Enter an integer: ";
    // Read user input from standard input stream
    cin >> number;

    string result = (number % 2 == 0) ? "Even" : "Odd";

    // Output formatted data to console stream
    cout << number << " is " << result << "." << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 10: PrintMonthSwitch

Problem Description & Objective:

Implementation of PrintMonthSwitch

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to print the month of the year based on a number (1-12) input by the user.
 */

```

```

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *   - `switch-case` Selection Statement with `break`.
 *
 * 2. WHY IT IS USED?
 *   - `switch` provides direct jump table dispatch for discrete integer key matching.
 *
 * 3. HOW IT IS USED?
 *   - `switch (month)` handles cases 1 through 12, with a `default:` for invalid inputs.
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input integer `month`.
 * *   Step 2: Jump to matching `case` 1..12.
 * *   Step 3: Print month name and `break`.
 * *   Step 4: If no match, execute `default:` block.
 * =====
 */

```

```

#include <iostream> // Header for standard console stream I/O

```

```

using namespace std; // Standard namespace

```

```

// --- Main Execution Entry Point ---
int main() { // Program start
    // Declare integer variable for month number
    int month;

    // Read month number input from user
    cout << "Enter month number (1-12): ";
    // Read user input from standard input stream
    cin >> month;

    // Switch statement jumps directly to matching integer case
    switch (month) {
        // Output formatted data to console stream
        case 1: cout << "January" << endl; break; // Case 1: January
        // Output formatted data to console stream
        case 2: cout << "February" << endl; break; // Case 2: February
        // Output formatted data to console stream
        case 3: cout << "March" << endl; break; // Case 3: March
        // Output formatted data to console stream
        case 4: cout << "April" << endl; break; // Case 4: April
        // Output formatted data to console stream
        case 5: cout << "May" << endl; break; // Case 5: May
        // Output formatted data to console stream
        case 6: cout << "June" << endl; break; // Case 6: June
        // Output formatted data to console stream
        case 7: cout << "July" << endl; break; // Case 7: July
        // Output formatted data to console stream

```

```
case 8: cout << "August" << endl; break;    // Case 8: August
// Output formatted data to console stream
case 9: cout << "September" << endl; break; // Case 9: September
// Output formatted data to console stream
case 10: cout << "October" << endl; break;   // Case 10: October
// Output formatted data to console stream
case 11: cout << "November" << endl; break;  // Case 11: November
// Output formatted data to console stream
case 12: cout << "December" << endl; break;  // Case 12: December
default:                                     // Default branch for values outside 1-12
    // Output formatted data to console stream
    cout << "Invalid month number! Please enter 1-12." << endl;
    break;
}

// Finish program execution
return 0;
}
```

Problem 11: ScoreCategoryTO

Problem Description & Objective:

Implementation of ScoreCategoryTO

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to Based on a student's score, categorize as "High", "Moderate", or "Low"
 * using the ternary operator (e.g., High for scores > 80, Moderate for 50-80, Low for < 50).
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Nested Ternary Operator (`cond1 ? res1 : (cond2 ? res2 : res3)`).
 *
 * 2. WHY IT IS USED?
 *    - Allows multi-level conditional evaluation in a single assignment statement.
 *
 * 3. HOW IT IS USED?
 *    - `(score > 80) ? "High" : ((score >= 50) ? "Moderate" : "Low")`
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Read `score`.
 *    Step 2: Evaluate nested ternary logic.
 *    Step 3: Output string category.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    double score;

    // Output formatted data to console stream
    cout << "Enter student score: ";
    // Read user input from standard input stream
    cin >> score;

    string category = (score > 80) ? "High" : ((score >= 50) ? "Moderate" : "Low");

    // Output formatted data to console stream
    cout << "Score Category: " << category << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 12: SimpleCalculatorSwitch

Problem Description & Objective:

Implementation of SimpleCalculatorSwitch

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to create a simple calculator that uses a switch statement to perform
 * basic arithmetic operations like addition, subtraction, multiplication, and division.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Character-based `switch` Statement and Basic Arithmetic.
 *
 * 2. WHY IT IS USED?
 *    - Evaluates menu-driven character operators `'+', '-', '*', '/'`.
 *
 * 3. HOW IT IS USED?
 *    - `switch(op)` branches to corresponding arithmetic operation block.
 *
 * 4. ALGORITHM EXPLANATION:
 *    Step 1: Input operator character `op` and numbers `num1`, `num2`.
 *    Step 2: Branch based on `op`.
 *    Step 3: Check for division by zero before dividing.
 *    Step 4: Display computed result.
 * =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    char op;
    double num1, num2;

    // Output formatted data to console stream
    cout << "Enter operator (+, -, *, /): ";
    // Read user input from standard input stream
    cin >> op;

    // Output formatted data to console stream
    cout << "Enter two numbers: ";
    // Read user input from standard input stream
    cin >> num1 >> num2;

    // Switch jump-table selection statement
    switch (op) {
        case '+':
            // Output formatted data to console stream
            cout << num1 << " + " << num2 << " = " << (num1 + num2) << endl;
            break;
        case '-':
            // Output formatted data to console stream
            cout << num1 << " - " << num2 << " = " << (num1 - num2) << endl;
            break;
    }
}

```

```

    case '*':
        // Output formatted data to console stream
        cout << num1 << " * " << num2 << " = " << (num1 * num2) << endl;
        break;
    case '/':
        // Evaluate conditional decision logic
        if (num2 != 0)
            // Output formatted data to console stream
            cout << num1 << " / " << num2 << " = " << (num1 / num2) << endl;
        // Alternative conditional branch
        else
            // Output formatted data to console stream
            cout << "Error! Division by zero." << endl;
        break;
    default:
        // Output formatted data to console stream
        cout << "Invalid operator!" << endl;
        break;
}

// Return zero code indicating successful program finish
return 0;
}

```

6. Exam Scoring Points & Memory Revision 🎯

★ Must-Write Points

1. switch statements utilize compiler jump tables for O(1) multi-way branch efficiency.
2. Nested if-else handles complex multi-condition interval checks (e.g. Grade calculations, Leap years).
3. Modern C++ supports if statements with initializer: `if (int x = getVal(); x > 0) { ... }`.
4. break statement terminates switch block immediately; default handles fallback cases.
5. Logical operators (&&, ||) combine relational conditions with short-circuit efficiency.

⚡ Quick Recall

Branch Evaluation -> Jump Table Switch -> Ternary Inline -> Scope Bound Initializer